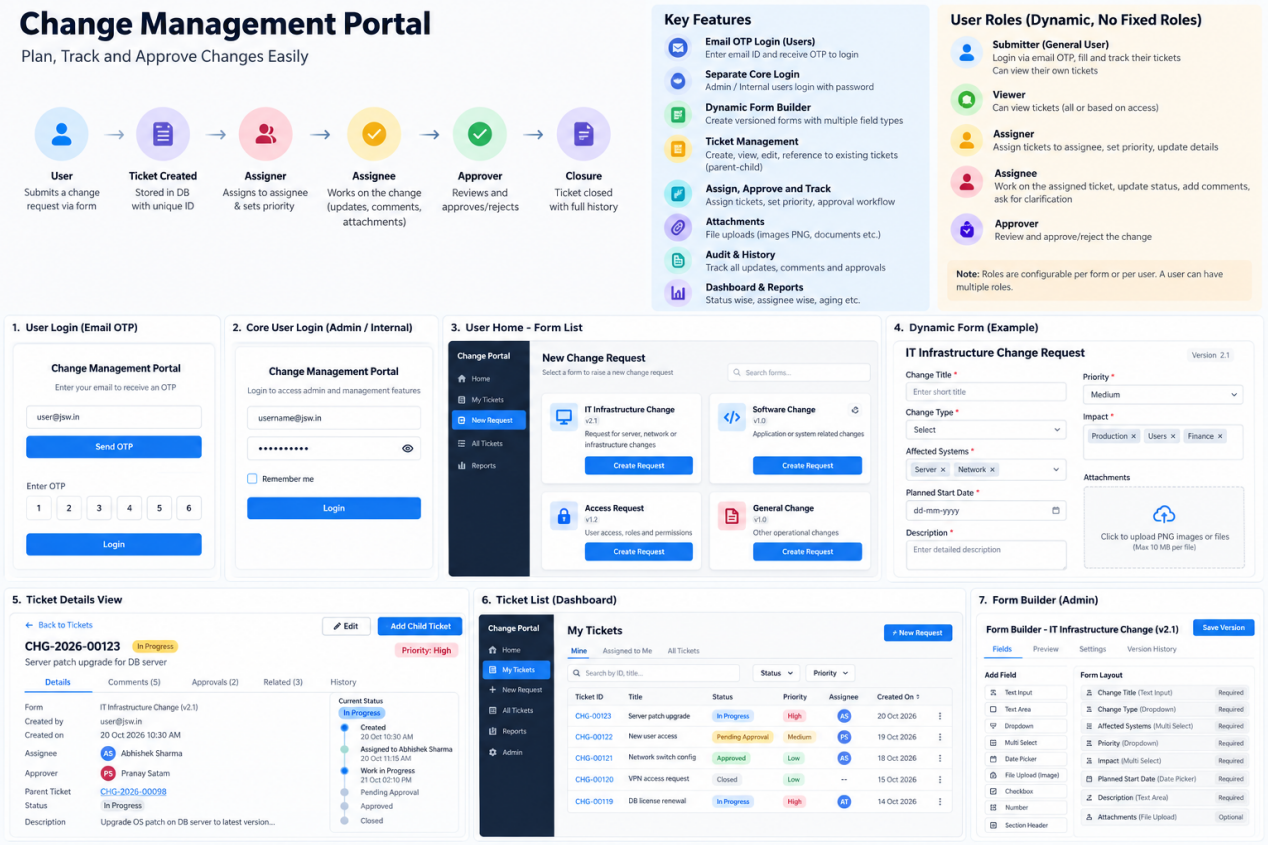
CHANGE MANAGEMENT PORTAL

Product Plan, Feature Set & UX Design Concept

Purpose

A configurable internal portal for submitting, assigning, reviewing, approving and tracking change requests — built with Flask and SQLite.



Design direction: clean enterprise UI, clear ticket states, dynamic form configuration, explicit ownership and a complete audit trail.

Technology baseline: Flask · SQLAlchemy · SQLite · Bootstrap 5 · JavaScript · SMTP / corporate mail service

1. Executive Overview

The portal should be form-driven, version-controlled and permission-configurable. Administrators should be able to add forms, change field options and configure approval flows without requiring a code deployment.

Design principles

- **Two login experiences:** email OTP for requesters and a separate core-user login for operational users.
- **No fixed operational roles:** permissions are configurable per form, ticket, department or scope.
- **Versioned forms:** published form versions remain attached to tickets created with that version.
- **Shared ticket engine:** assignment, approval, parent-child relationships and audit history work consistently across forms.
- **Server-side security:** every API operation checks permissions; hiding a button in the UI is not access control.

Core workflow

1. Login Email OTP or core-user login	**2. Submit** Select a published form and create ticket	**3. Triage** Viewer reviews; assigner sets priority and owner
4. Execute Assignee works, comments and uploads evidence	**5. Approve** Configured approvers approve, reject or return	**6. Close** Retain history and relationships

Recommended MVP scope

- Authentication, user management and configurable permission profiles.
- Dynamic form builder with draft/publish/version history.
- Ticket creation, edit, assignment, priority, comments and attachments.
- Approval stages and audit trail.
- Parent-child, related, duplicate and follow-up ticket links.

2. Feature Set

Module	Feature scope
Authentication & access	Requester email OTP; separate core login; user activation/revocation; throttling; session expiry; optional email-domain allowlist.
Dynamic form builder	Create, edit, duplicate, publish and archive forms; field types; validations; conditional visibility; preview; immutable published versions.
Ticket management	Unique ticket IDs; drafts; submit/resubmit; status, priority, due date; comments; attachments; filters; controlled edit and reopen.
Assignment & workflow	Assign/reassign; configurable status transitions; work notes; clarification requests; due dates; optional SLA and escalation rules.
Approval engine	One or many approval stages; sequential or parallel approvals; approve/reject/return; comments; conditional approval; reapproval after material changes.
Ticket relationships	Parent-child; related; duplicate-of; follow-up-to; create from existing ticket; selected field carry-forward; parent/child status visibility.
Audit & notifications	Field-level change history; actor and timestamp; assignment/approval events; email notifications; attachment metadata and access logging.
Dashboard & reports	Open/overdue/pending-approval counts; views by form, priority and assignee; resolution and approval time; exports. Phase 2 enhancements.

Recommended form field types

Field type	Configuration
Text / Text area	Length limits, placeholder, regex, multiline
Dropdown / Multi-select	Configured options, defaults, required selection
Date / Date-time	Date ranges, time constraints
Number / Checkbox	Min/max, decimal precision, boolean
Image upload	PNG/JPEG/WebP allowlist, size and count limits
File upload	MIME allowlist, size limit, private storage
Section / Conditional	Group fields; show/require fields based on earlier answers

3. Configurable Roles & Permissions

Do not hardcode Viewer, Assigner, Assignee or Approver as exclusive application roles. Treat these as editable permission profiles that administrators can create and assign at different scopes.

Example profile	Typical permissions
Requester	Create requests; view own or explicitly shared tickets
Viewer	Read permitted tickets, comments and history
Assigner	Assign/reassign; set priority and due date
Assignee	Update work status; add comments and evidence
Approver	Approve, reject or return a request
Form Administrator	Configure forms, versions and workflow rules
System Administrator	Manage accounts, authentication and global settings

Example permission keys

- `ticket.view`, `ticket.create`, `ticket.edit`, `ticket.assign`, `ticket.set_priority`
- `ticket.approve`, `ticket.reopen`, `ticket.link`, `form.publish`, `audit.view`
- Scope permissions by form, ticket, department or global access; enforce them on every backend endpoint.

4. Dynamic Form Builder & Versioning

Store form definitions as JSON with stable field IDs, field types, labels, options, validation rules, ordering, conditions and version metadata. The frontend renders the form; the server validates submissions against the same published schema.

- Administrator creates a draft and previews it.
- Publish creates an immutable version.
- New tickets use the latest published version.
- Existing tickets keep their original form-version reference and remain renderable.
- Changes to critical fields can trigger reapproval based on configured policy.

Illustrative JSON definition

```
{
  "form_key": "INFRA_CHANGE",
  "version": 2,
  "fields": [
    {"id": "change_title", "label": "Change Title",
     "type": "text", "required": true},
    {"id": "affected_systems", "label": "Affected Systems",
     "type": "multi_select",
     "options": ["Server", "Network", "Database"],
     "required": true},
    {"id": "evidence", "label": "Supporting Image",
     "type": "image", "accept": ["image/png"],
     "required": false}
  ]
}
```

5. Ticket Relationships & Editing

A new ticket may be a child of, related to, a duplicate of, or a follow-up to an older ticket. Keep each ticket independently auditable; do not silently merge records or overwrite the original submission.

Relationship	Expected behavior
Parent-child	Work is broken into dependent or subordinate tasks; optionally display child status on parent.
Related	Connect independent work items without implying ownership.
Duplicate-of	Identify duplicates while preserving the duplicate's record and audit trail.
Follow-up-to	Create a new request using selected details from an earlier ticket.

Editing rules

- Allow edits based on permissions and ticket state; record old/new values, actor, time and reason where required.
- If a ticket is edited while another user has it open, detect conflicts instead of silently overwriting changes.
- Changing an approved material field can invalidate prior approval or trigger reapproval.
- Reopening a ticket should require permission and be recorded in the audit log.

6. Suggested Screen Inventory

Screen	Key elements
Requester login	Email entry, OTP verification, resend timer and clear error states.
Core login	Separate sign-in route, secure password handling and account recovery policy.
Requester home	Published form catalogue, search, create request and My Tickets.
Ticket list	Tabs for Mine, Assigned to Me, All Permitted; filters for status, priority, form and date.
Ticket details	Form data, current status, owner, comments, attachments, approvals, relationships and history.
Form builder	Field palette, layout, field settings, preview, publish and version history.
Workflow settings	Assignment rules, approver stages, permission profiles and status transitions.
Reports	Queue health, overdue work, approval latency and exports.

7. Technical Architecture

Frontend: Jinja templates, Bootstrap 5 and JavaScript. Backend: Flask Blueprints and REST endpoints. SQLAlchemy manages persistence; Alembic migrations manage schema changes. SMTP or an approved mail API sends OTPs and workflow notifications.

Frontend
Requester portal · Core workspace · Dynamic form renderer · Admin builder

Flask application / API
Authentication · Authorization · Forms · Tickets · Approvals · Files · Notifications

Persistence & services
SQLite relational data + JSON form definitions · Private file storage · Corporate email service

Suggested database tables

Domain	Tables
Identity	users, otp_challenges, core_credentials
Forms	forms, form_versions, form_access
Permissions	permission_profiles, user_profile_assignments
Tickets	tickets, ticket_values, ticket_relationships, ticket_comments
Workflow	approval_steps, ticket_approvals, ticket_history
Files & notifications	attachments, notifications

SQLite guidance

- Enable foreign-key constraints, WAL mode, transactions and a busy timeout.
- Store uploads outside the public web root; store only metadata and server-generated storage paths in SQLite.
- SQLite is suitable for a modest internal deployment; consider PostgreSQL or SQL Server if concurrent writes and usage grow substantially.
- Use database migrations, automated backups and tested restore procedures.

8. Security & Governance

- **OTP:** random codes, short expiry, single use, hashed storage, attempt limits and resend throttling.
- **Core login:** strong password hashing, rate limiting, session expiry and optional corporate SSO later.
- **Authorization:** verify permissions on every API route and file download; do not rely on frontend hiding.
- **Uploads:** validate file signature and MIME type, allowlist extensions, enforce size limits, use generated filenames and scan files where possible.
- **Approvals:** record approver, decision, timestamp, comments and the ticket data version reviewed.
- **Audit:** record material field changes, assignment changes, workflow transitions, approval actions and reopens.
- **Operations:** backup database and files together; test restore, define retention and monitor email delivery.

9. Implementation Roadmap

Phase	Deliverables
Phase 1 · Foundation	Flask skeleton, migrations, OTP and core login, user management, permission profiles.
Phase 2 · Configuration	Dynamic form builder, field types, preview, draft/publish and version history.
Phase 3 · Operations	Ticket CRUD, assignment, priority, comments, attachments, search and ticket relationships.
Phase 4 · Governance	Approval engine, notifications, audit history, reapproval rules and escalation basics.
Phase 5 · Production	Security and workflow tests, backups, restore tests, reports, deployment hardening and UAT.

Planning estimate: approximately 4–6 weeks for an initial usable release by one developer, assuming requirements and mail access are available. Complex approvals, security review and production deployment may extend the timeline.

10. MVP Acceptance Criteria

- Requester can verify an email OTP and submit a dynamically rendered form.
- Administrator can publish a new form version without a code deployment.
- Existing tickets remain tied to their original form version.
- Core users receive configurable permissions and can assign, prioritize and act on permitted tickets.
- Parent-child and related ticket links work without losing independent histories.
- Approvals record decisions and reject unauthorized or duplicate actions.
- Unauthorized users cannot edit tickets or download restricted attachments.
- Material changes appear in audit history and database/file backup restoration is verified.

Recommended build order

Build the shared ticket engine, configurable permission system and versioned form renderer first. Assignment, approval and ticket relationships should sit on top of these shared foundations rather than being separately implemented for every form.

Document prepared as a product-planning and UX concept. Screen visuals are illustrative references, not production UI or implemented functionality.